

HCPS: Heuristic Candidate-space Policy Search for Agile Earth Observation Satellite Scheduling

Pengfei Gao¹, Benkui Zhang², Yajing Xu¹, Jinxiao Wang², Zhiyuan Yu²,
Kangning Du³, and Bo Xiao^{1*}

¹ Beijing University of Posts and Telecommunications, Beijing 100101, China
xiaobo@bupt.edu.cn

² Key Lab of Target Cognition and Application Technology, Aerospace Information
Research Institute, CAS, Beijing 100101, China

³ Key Laboratory of Information and Communication Systems, Ministry of
Information Industry, Beijing Information Science and Technology University, Beijing
100101, China

Abstract. Many real-world combinatorial optimization tasks are constrained constructive processes with expensive per-step fitness evaluation. Population-based methods often suffer from high evaluation cost, while single-solution methods may lack exploration. We propose Heuristic Candidate-space Policy Search (HCPS), a hybrid framework that combines heuristic candidate-space shaping with policy-based sequential selection. At each decision step, HCPS applies feasibility-aware Top- K pruning to compress the primitive action space and then uses Proximal Policy Optimization (PPO) to optimize long-horizon utility on the reduced candidate set. We provide complexity analysis showing that Top- K pruning can substantially reduce computation while keeping quality loss limited when high-value actions are retained with high probability. Experiments on agile Earth observation satellite scheduling as a representative testbed demonstrate a strong quality-time trade-off and support the framework’s generalizability.

Keywords: Heuristic Candidate-space Policy Search · Hybrid Search · Agile Earth Observation Satellite Scheduling Problem · Reinforcement Learning

1 Introduction

Agile Earth Observation Satellites (AEOS) can adjust attitude along roll, pitch, and yaw axes, achieving significantly longer observation windows than traditional satellites. This flexibility introduces complex constraints (e.g., attitude transition times), making the Agile Earth Observation Satellite Scheduling Problem (AEOSSP) a challenging combinatorial optimization task [6, 26, 9]. A key characteristic of AE OSSP—and many real-world scheduling problems—is that evaluating a single action (assigning an observation) is computationally expensive:

* Corresponding author

each candidate requires checking geometric visibility, attitude feasibility, resource constraints, and conflicts with scheduled tasks. This per-step cost forces a fundamental trade-off between efficiency and solution quality.

Heuristic and metaheuristic methods have long been the dominant paradigm. Constructive heuristics [22, 14, 13] first defined AEOSSP and proposed greedy, dynamic programming, constraint programming, and local search; later works improved them with limited backtracking [24], priority ordering [27], and complex network theory [25]. Metaheuristics include evolutionary algorithms (genetic algorithms [15, 16], ant colony optimization [5, 4]) and single-point search (ALNS [18, 11], GRILS [20]). They escape local optima but incur high computational cost due to many feasibility checks.

Recently, reinforcement learning (RL) methods have been explored: A3C for online scheduling [23], data-driven parallel scheduling with C-NEAT [7], end-to-end pointer networks [2, 1], Q-learning [12], RL-guided genetic algorithm [21], and learning-to-construct [3]. Despite advances, most RL methods either operate directly on the large primitive action space (all targets) or adopt a two-stage “sequence-then-generate” pipeline where feasibility is handled after policy decisions. Both suffer from exploration inefficiency or costly repair.

Action space reduction has succeeded elsewhere: AlphaGo [10] uses policy-guided MCTS to expand only top moves; quadruped robots use kinematic priors [8]. However, these either rely on learned networks or are tailored to continuous control, not directly transferable to combinatorial optimization. In combinatorial optimization, domain heuristics are often used only as pre-filters or reward shaping, not structurally integrated.

In this work, we treat heuristics as structural shapers of the search space through two complementary designs: a feasibility-aware Top- K heuristic pruning that compresses the primitive action space into a compact candidate set, and a structured feature extraction module that enriches each candidate. Proximal Policy Optimization (PPO) trains a policy that selects actions from this candidate space to maximize long-horizon utility. Heuristics encode feasibility and immediate reward, whereas RL focuses on sequential strategies within a structured action space.

We instantiate this framework as HCPS (Heuristic Candidate-space Policy Search) and evaluate it on AEOSSP. Our contributions are:

- A hybrid optimization paradigm combining heuristic candidate-space shaping with policy-based selection, shifting heuristics from pre-filtering to structural guidance.
- A feasibility-aware Top- K pruning mechanism with theoretical complexity analysis, showing substantial computation reduction while preserving high-quality actions with high probability.
- Extensive experiments on AEOSSP demonstrating HCPS outperforms pure heuristics, population-based methods, and existing RL-with-heuristic baselines, achieving superior quality–time trade-off.

The remainder of this paper is organized as follows. Section 2 formally defines the AEOSSP. Section 3 presents the HCPS framework in detail. Section 4

describes the experimental setup and analyzes the results. Section 5 concludes the paper and outlines future research directions.

2 Problem description and formulation

2.1 Description of the AEOSSP

In line with related studies [18, 19], this paper introduces the following assumptions for the AEOSSP.

1. The problem assumes the availability of sufficient ground stations and relay satellites.
2. Each AEOS can observe only one target at a time.
3. The problem assumes that the satellite has sufficient energy and storage capacity.
4. Each target can be observed at most once, known as the multi-orbit uniqueness constraint.

In addition to the above assumptions, the constraints that must be satisfied are as follows:

1. Observation windows (OWs) must lie within their visible time windows (VTWs) and meet the minimum duration.
2. Sufficient transition time between consecutive OWs for attitude maneuvering.

Objective: maximize total observation profit. Table 1 lists main symbols.

Table 1. Main notation

<i>Symbol</i>	<i>Description</i>
T	Set of targets
T_i	Target i
p_i	Profit of target T_i
d_i	Required observation duration of target T_i
O	Set of satellite orbits
O_j	Orbit j
$[st_j, et_j]$	Start and end time of orbit O_j
VTW_i^j	Visible time window of orbit O_j for target T_i
$[vs_i^j, ve_i^j]$	Start and end time of VTW_i^j
vt_i^j	Optimal (central) time of VTW_i^j
OW_i^j	Selected observation window from VTW_i^j
os_i^j	Start time of OW_i^j
$\phi_i^j, \theta_i^j, \psi_i^j$	Roll, pitch, yaw angles of OW_i^j
$\Delta g(OW_m^j, OW_n^j)$	Total angle change between two OWs
$\text{tran}(\Delta g)$	Required transition time for angle change Δg

2.2 Formulation of the AEOSSP

Two types of decision variables are defined:

1. Binary variable x_i^j : equals 1 if target T_i is observed in orbit O_j , otherwise 0.
2. Real variable os_i^j : start time of the observation window OW_i^j when $x_i^j = 1$.

The mathematical model is as follows:

$$\text{Maximize } \sum_{i \in T} \sum_{j \in O} p_i \cdot x_i^j \quad (1)$$

$$\text{subject to } \sum_{j \in O} x_i^j \leq 1, \quad \forall i \in T \quad (2)$$

$$x_i^j \in \{0, 1\}, \quad os_i^j \in \mathbb{R} \quad (3)$$

$$x_i^j \cdot (os_i^j - vs_i^j) \geq 0, \quad \forall i \in T, j \in O \quad (4)$$

$$x_i^j \cdot (ve_i^j - d_i - os_i^j) \geq 0, \quad \forall i \in T, j \in O \quad (5)$$

$$\begin{aligned} os_m^j + d_m + \text{tran}(\Delta g(OW_m^j, OW_n^j)) &\leq os_n^j, \\ \forall j \in O, \forall m, n \in T \text{ with } x_m^j = x_n^j = 1, \\ \text{and } OW_m^j \text{ immediately precedes } OW_n^j \end{aligned} \quad (6)$$

Constraint (2) ensures each target is observed at most once. Constraints (4) and (5) enforce that the observation window lies within the visible time window and meets the minimum duration. Constraint (6) guarantees sufficient transition time between two consecutive observations on the same satellite orbit.

Transition time follows the piecewise function from [18]:

$$\text{tran}(\Delta g) = \begin{cases} 11.66, & \Delta g \leq 10 \\ a_1 + \Delta g/v_1, & 10 < \Delta g \leq 30 \\ a_2 + \Delta g/v_2, & 30 < \Delta g \leq 60 \\ a_3 + \Delta g/v_3, & 60 < \Delta g \leq 90 \\ a_4 + \Delta g/v_4, & \Delta g > 90 \end{cases} \quad (7)$$

with parameters $(v, a) = (1.5, 5), (2, 10), (2.5, 16), (3, 22)$ for the AS-01 satellite.
 $\Delta g(OW_m^j, OW_n^j) = |\phi_m^j - \phi_n^j| + |\theta_m^j - \theta_n^j| + |\psi_m^j - \psi_n^j|$.

3 Methodology: Heuristically-Shaped Policy Optimization

3.1 MDP Formulation and HCPS Framework

The AEOSSP can be formulated as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, where:

- $\mathcal{S}$: state space, each state s encoding a partial schedule.

- $\mathcal{A}$: action space, each action a corresponding to a feasible VTW insertion.
- $\mathcal{P}$: deterministic transition, $P(s_{t+1}|s_t, a_t) = 1$ if a_t leads to s_{t+1} .
- $\mathcal{R}$: immediate reward r_t , set as the profit of the completed target.

A policy π maximizes the expected cumulative reward $\mathbb{E}[\sum \gamma^t r_t]$.

The primitive action set $\mathcal{A}(s_t)$ is huge (e.g., 5800 VTWs for 1000 targets and 6 satellites), making direct optimization prohibitive. Therefore, HCPS restructures this MDP into a two-layer process: first heuristic candidate-space shaping, then policy optimization on the reduced set.

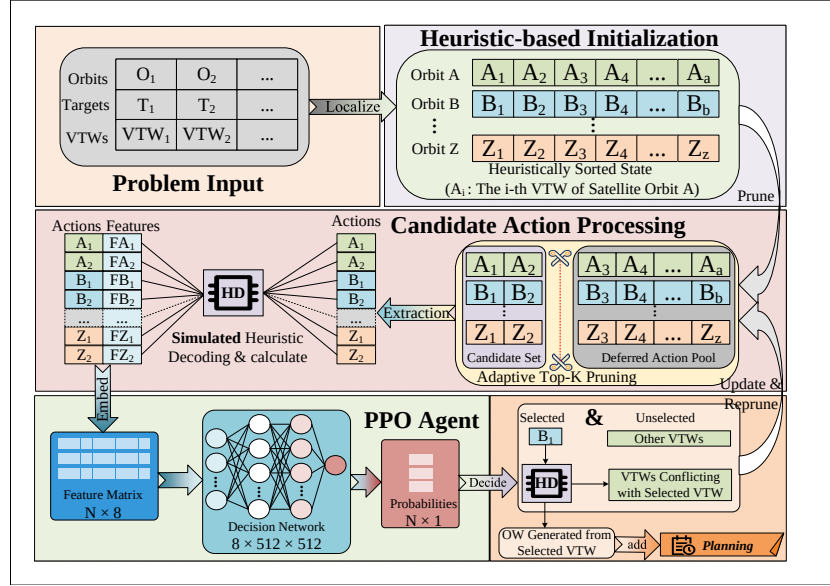


Fig. 1. Overall framework of HCPS. The heuristic component consists of three sub-modules: Heuristic-based Initialization (HBI), Candidate Action Processing (CAP), and Heuristic Decoder (HD). The policy-based selection uses Proximal Policy Optimization (PPO). HBI performs conflict-driven orbit splitting and sorting; CAP performs adaptive Top- K pruning and deep feature extraction; HD handles simulation and execution. The PPO agent selects actions from the reduced candidate set C_t .

The overall architecture of HCPS is shown in Fig. 1. The framework consists of two components: Heuristic Candidate-space Shaping and Policy-based Selection. The heuristic component comprises three sub-modules:

- **HBI:** Preprocesses the raw input via conflict-driven orbit splitting and intra orbit sorting (Earliest Deadline First [17] and start-time priority). This produces a structured initial state that respects temporal order.
- **CAP:** At each decision step, performs adaptive Top- K pruning on the candidate VTWs within each orbital group, where the number of retained can-

didates per orbit is dynamically adjusted based on current resource competition. For each retained VTW, the HD is invoked in simulation mode to perform a tentative insertion, and deep features are extracted from the resulting post-decoding state. These feature vectors are then passed to the PPO agent.

- **HD**: Converts a selected VTW into an executable observation window (OW) and updates the environment. It operates in two modes: simulation returns the resulting feature vector without altering the live schedule; execution inserts the OW, computes the reward, and removes the scheduled target and conflicting VTWs.

The PPO agent receives the normalized feature vectors of finite candidates from CAP and outputs a probability distribution over them. It samples an action according to this distribution, and the chosen VTW is passed to HD for execution. The agent is trained to maximize cumulative scheduling reward using the proximal policy optimization algorithm.

3.2 Heuristic Candidate-Space Shaping

Heuristic-Based Initialization (HBI) HBI preprocesses the raw input through two operations: conflict-driven orbit splitting and heuristic intra-orbit ordering. For orbit O_j , compute the maximum viewing angle change among its VTWs:

$$\Delta g_{\max}^j = \max_{m,n \in T} (|\phi_m^j - \phi_n^j| + |\theta_m^j - \theta_n^j| + |\psi_m^j - \psi_n^j|), \quad (8)$$

and the corresponding maximum transition time $\tau_{\max}^j = \text{tran}(\Delta g_{\max}^j)$. The orbit is recursively split at any gap Δt_k between consecutive VTWs satisfying $\Delta t_k > \tau_{\max}^j$, yielding independent sub-orbits with no cross-conflict.

Within each virtual orbit, assign a priority score to each VTW:

$$G_i^j = \alpha \left(1 - \frac{ve_i^j - st_j}{et_j - st_j} \right) + \beta \left(1 - \frac{vs_i^j - st_j}{et_j - st_j} \right), \quad (9)$$

The first term implements the Earliest Deadline First (EDF) principle (earlier ve_i^j gives higher score), while the second term favors VTWs with earlier start times. We set $\alpha = 1$ and $\beta = 0.2$ to prioritize deadline pressure while still considering start time. VTWs are sorted descending by G_i^j to form an initial sequence, providing a temporally coherent state for subsequent modules.

Candidate Action Processing (CAP) CAP handles the selection of promising VTWs at each decision step through two sequential operations: adaptive Top- K pruning and deep feature extraction.

Let A_t be the primitive feasible action set at state s_t . Instead of using a fixed K , we adapt the pruning ratio per orbit based on its current resource competition intensity at decision step t . For orbit O_j , let $N_{VTW}^j(t)$ be the number

of remaining VTWs in that orbit at time t . Define the expected occupancy per VTW as $\bar{t}_{\text{occ}} = d_{\text{avg}} + \bar{\tau}$, where d_{avg} is the average required observation duration and $\bar{\tau}$ is the expected transition time (derived from historical data or a fixed prior). The total estimated time required to schedule all remaining VTWs is $T_{\text{need}}^j(t) = N_{VTW}^j(t) \cdot \bar{t}_{\text{occ}}$, and the remaining available time in the orbit is $T_{\text{avail}}^j(t) = et_j - st_j - t_{\text{used}}(t)$, where $t_{\text{used}}(t)$ is the total time already occupied by scheduled observations in that orbit. The resource competition ratio at step t is defined as:

$$r_j(t) = \frac{T_{\text{need}}^j(t)}{T_{\text{avail}}^j(t)}. \quad (10)$$

A larger $r_j(t)$ indicates tighter resource competition (more VTWs competing for limited time), which demands a larger candidate set to allow careful selection. We set the adaptive $K_j(t)$ for orbit O_j at step t as:

$$K_j(t) = \max(2, \lfloor r_j(t) \rfloor), \quad (11)$$

where $\lfloor \cdot \rfloor$ denotes the floor function. This formulation ensures that $K_j(t)$ increases with competition intensity, retaining more candidates when resources are scarce, with a minimum of 2 to maintain a minimal level of choice.

We then sort all feasible candidate VTWs in A_t (i.e., VTWs that satisfy all constraints given the current partial schedule) descending by their priority scores G_i^j computed in Section 3.2. For each orbit O_j , only the top $K_j(t)$ candidates are kept:

$$C_t = \bigcup_j \text{TopK}_{a \in A_t^j} G_i^j, \quad |C_t| = \sum_j K_j(t) \ll |A_t|, \quad (12)$$

where A_t^j is the set of VTWs belonging to orbit O_j . PPO selects actions only within C_t . This shapes the search manifold using domain heuristics and allocates computation from breadth to depth, with $K_j(t)$ automatically increasing under tight resource conditions to enable more informed decisions.

After pruning, the Heuristic Decoder performs a *full decoding call* for each candidate $a \in C_t$: it converts the selected VTW into an executable OW, tentatively inserts it into a copy of the current schedule, and evaluates the consequences on all remaining VTWs in the same orbit. This process is computationally expensive because it involves OW generation, transition feasibility checks, and conflict propagation.

For each candidate, the following raw features are extracted from the simulation:

- **Transition time overhead:** the time $\text{tran}(\Delta g)$ required for the insertion, normalized by Tran_{max} (the maximum transition time in the instance).
- **Conflict count:** the number of VTWs that become infeasible due to the insertion.
- **Target kill count:** the number of targets that lose all their remaining VTWs (i.e., the insertion removes their last feasible opportunity).
- **Orbit resource pressure:** the ratio of remaining available time in the orbit to the number of remaining VTWs in that orbit.

- **Candidate rank:** the position of the VTW in the Top- $K_j(t)$ list (from 1 to $K_j(t)$), normalized by $K_j(t)$.
- **Remaining opportunities:** the number of VTWs still available for the same target across all orbits, normalized by the total number of VTWs of that target.

These raw features are then normalized within the candidate set C_t to provide relative comparisons, enabling the PPO agent to make decisions based on relative advantages rather than absolute magnitudes.

Let c_f be the cost of one full decoding call and c_h the heuristic scoring cost. In the primitive action space, naïvely evaluating all $|A_t|$ candidates would require $\mathcal{O}(|A_t|c_f)$ per step. With HCPS, the lightweight heuristic scoring first selects the $K_j(t)$ most promising VTWs per orbit, and only these $\sum_j K_j(t)$ candidates undergo the expensive decoding. Thus the per-step complexity becomes $\mathcal{O}(|A_t|c_h + (\sum_j K_j(t))c_f)$. Because $\sum_j K_j(t) \ll |A_t|$ and $c_h < c_f$, the number of costly full decoding calls is reduced substantially.

If a_t^* is the optimal primitive action and $p_t = \Pr(a_t^* \in C_t)$, the expected value loss is bounded by $\sum_{\tau=t}^T \gamma^{\tau-t}(1-p_\tau)\Delta_{\max}$; thus high retention probability limits quality degradation. The adaptive $K_j(t)$ ensures that p_t remains high especially when resource competition is intense, by retaining a sufficiently large candidate set for careful selection.

3.3 Policy Learning on Candidate Space

State and Action Representation The state is implicitly encoded in the feature vectors of the candidates in C_t ; the agent does not maintain an explicit global state representation. The action space is the candidate set C_t (size K), each action corresponding to selecting a specific VTW.

PPO Agent Architecture The PPO agent comprises two neural networks with an identical architecture. The input is an 8-dimensional feature vector per candidate, which includes the six features listed in Section 3.2 plus two additional features: target profit p_i and required duration d_i , both normalized within the candidate set. The actor network maps these features to selection probabilities via a shared feature extraction layer followed by a softmax output over the K candidates. The critic network first transforms each candidate’s features independently, then aggregates them via average pooling to produce a scalar state value estimate. Action selection follows $a_t \sim \pi_\theta(\cdot|s_t) = \text{softmax}(f_\theta(s_t))$. Training uses the standard PPO-clip objective with generalized advantage estimation (GAE).

Reward Function Design The reward components are designed based on the features extracted in CAP, providing immediate feedback aligned with the candidate evaluation criteria. Rewards are hierarchical to provide dense learning

signals while focusing on profit maximization (Table 2). Here, $Tran_{\min} = 11.6$ seconds and $Tran_{\max}$ is the maximum transition time in the current instance.

Table 2. Reward components.

Component	Formula	Weight
Target completion	$R_{comp} = \omega_1 p_i$	ω_1 (large)
Kill penalty	$R_{kill} = -\omega_2 \sum_i p_i k_i$	ω_2 (large)
Transition cost	$R_{time} = -\omega_3 \cdot \frac{tran - Tran_{\min}}{Tran_{\max} - Tran_{\min}}$	ω_3 (small)
Urgency	$R_{chance} = \omega_4 \left(1 - \frac{chance}{Chance_{\max}}\right)$	ω_4 (small)

Total reward: $R_{all} = R_{comp} + R_{kill} + R_{time} + R_{chance}$, with $\omega_1, \omega_2 \gg \omega_3, \omega_4$ to prioritize primary objectives. The specific weight values are provided in the experimental setup (Section 4).

3.4 Heuristic Decoder

HD converts selected VTWs into executable OWs and updates the environment. Algorithm 1 shows its dual-mode operation: *simulation* copies the schedule, inserts the candidate OW, and returns features; *execution* inserts the OW into the live schedule, computes reward, and updates state (removing scheduled target and conflicting VTWs).

Algorithm 1 Heuristic Decoder (HD)

- 1: **Input:** orbit index j , current schedule S , target set T , candidate VTW vtw , boolean *simulate*
 - 2: $S_j \leftarrow S[j]$ schedule of orbit j (list of scheduled OWs)
 - 3: **if** *simulate* **then**
 - 4: $ow \leftarrow \text{convertVTWtoOW}(vtw)$ generate OW from VTW, respecting attitude feasibility
 - 5: $vtw.ow \leftarrow ow$ store OW for potential execution
 - 6: $S'_j \leftarrow \text{copy}(S_j)$ create a copy of the current orbit schedule
 - 7: $S'_j \leftarrow \text{insertOW}(S'_j, ow)$ tentative insertion, checks transition constraints
 - 8: $features \leftarrow \text{extractFeatures}(S'_j, ow)$ compute feature vector for PPO
 - 9: **return** *features*
 - 10: **else**
 - 11: $ow \leftarrow vtw.ow$ retrieve precomputed OW (must have been set during simulation)
 - 12: $S_j \leftarrow \text{insertOW}(S_j, ow)$ permanently update the live schedule
 - 13: $reward \leftarrow ow.profit$ immediate reward equals target profit
 - 14: $\text{removeTarget}(T, vtw.target)$ mark target as observed
 - 15: $\text{removeConflictingVTWs}(S_j, ow)$ delete all VTWs that conflict with the new OW
 - 16: **return** *reward*
 - 17: **end if**
-

All helper functions (convertVTWtoOW, copy, insertOW, extractFeatures, removeTarget, removeConflictingVTWs) are standard schedule manipulation routines; their implementations are omitted for brevity.

4 Experiment

This section systematically analyzes the performance of the proposed HCPS framework. All training experiments were conducted on an NVIDIA A100 GPU, with code implemented in Python. Inference times (the solving times reported in the tables) are in seconds. Training on the 1000-target scale (six satellites) takes 3.5 hours on a single A100 GPU. For testing and evaluation, all experiments were run on an Intel i7-14700K processor platform equipped with 32 GB of RAM.

4.1 Dataset

Following prior studies [19, 18], the observation missions consist of point targets uniformly generated within China (3°N–53°N, 74°E–133°E). The orbital parameters of the six satellites used in the experiments are provided in the supplementary material (Table S1)⁴. Fifteen problem scales are evaluated, with the number of targets ranging from 100 to 1500 in steps of 100. Each target requires a uniformly random observation duration between 15 and 30 seconds, and each target is assigned a random profit uniformly drawn from [1, 10].

For each problem scale from 100 to 1500 in steps of 100, we generate 1000 independent instances for training and 20 separate instances for testing. All reported results in the following sections are statistical summaries over these 20 test instances. The primary quality metric is the target completion rate F_s , defined as:

$$F_s = \frac{\sum_{n \in N} f_n}{\sum_{n \in N} P_n}, \quad (13)$$

where f_n is the total profit obtained for instance n and P_n is the sum of profits of all targets in that instance.

4.2 Effect of Top- K Pruning

The parameter K in the Candidate Action Processing (CAP) module controls the number of candidate VTWs retained after heuristic pruning. We evaluate fixed K values {1, 3, 5, 10} and the proposed adaptive K strategy (denoted as K_{ada}) on three representative problem scales (700, 1000, 1500 targets). Fig. 2 shows the training convergence patterns for the fixed K values and the adaptive K .

Smaller fixed K values lead to faster initial convergence due to the reduced action space, which simplifies credit assignment. Larger fixed K values ($K = 5$) exhibit slower but more sustained improvement initially, yet excessively large K

⁴ Available at <https://github.com/Gaopf1016/hcps-supplementary>.

can hinder convergence: as shown in Table 3, an overly large action space may introduce redundant candidates and complicate policy learning. The value loss patterns in Fig. 2 further confirm that smaller K yields more stable training, while larger K introduces higher variance.

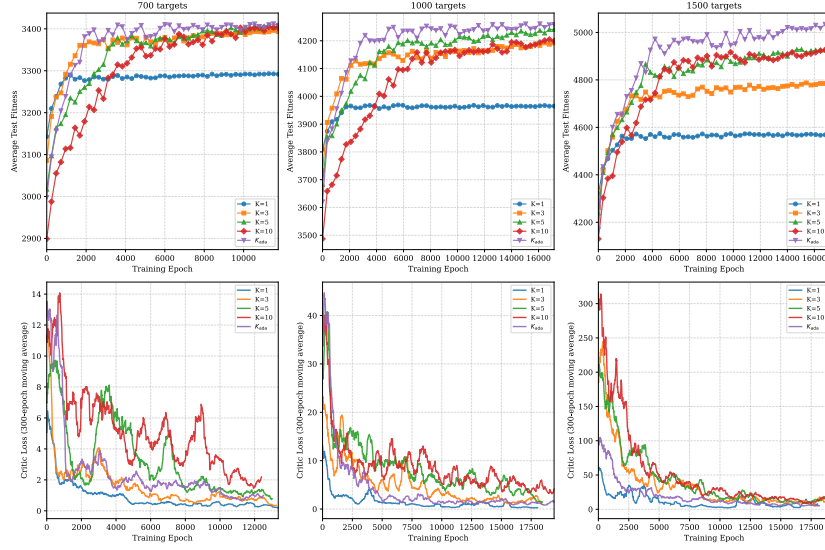


Fig. 2. Training convergence for different K values across three scales. The top row shows the average episode reward on test instances; the bottom row shows the critic loss during training. Each column corresponds to a specific problem scale (700, 1000, 1500 targets).

Table 3 quantifies the quality–time trade-off for fixed K and the adaptive K strategy. The adaptive K strategy achieves the highest completion rates on 1000 and 1500 targets, and is comparable to the best fixed K on 700 targets. More importantly, it does so with significantly lower computational cost than the best-performing fixed K values. This efficiency stems from the fact that larger K values increase computational time, while the adaptive mechanism dynamically shrinks the candidate set when resources are tight, reducing the per-step action space and accelerating decision making. Based on these results, we adopt the adaptive K strategy for all subsequent experiments.

Table 3. Performance comparison for different fixed K and adaptive K (mean $\pm$ std over 20 instances per scale).

K	700 targets		1000 targets		1500 targets	
	F_s (%)	Time (s)	F_s (%)	Time (s)	F_s (%)	Time (s)
1	85.52 $\pm$ 2.23	2.71 $\pm$ 0.32	72.10 $\pm$ 2.36	4.27 $\pm$ 0.21	55.38 $\pm$ 2.01	9.65 $\pm$ 0.65
3	88.16 $\pm$ 1.02	5.13 $\pm$ 0.28	76.22 $\pm$ 2.04	9.52 $\pm$ 0.24	58.05 $\pm$ 1.27	17.87 $\pm$ 0.72
5	88.57 $\pm$ 1.79	8.44 $\pm$ 0.31	77.04 $\pm$ 1.71	13.95 $\pm$ 0.35	59.84 $\pm$ 1.05	28.03 $\pm$ 0.89
10	88.49 $\pm$ 1.08	15.92 $\pm$ 0.44	76.35 $\pm$ 1.62	22.23 $\pm$ 0.58	59.93 $\pm$ 0.93	39.76 $\pm$ 1.23
K_{ada}	88.52 $\pm$ 1.25	6.41 $\pm$ 0.27	77.39 $\pm$ 1.82	11.43 $\pm$ 0.42	60.95 $\pm$ 0.85	23.49 $\pm$ 0.76

4.3 Comparison with State-of-the-Art Algorithms

We compare HCPS with four state-of-the-art baselines: GRILS [20], A-ALNS [11], GA, and RL-GA [21]. All algorithms are evaluated on the same dataset and hardware, with parameters tuned to their optimal settings as reported in the original papers. For HCPS, we adopt the adaptive Top- K pruning strategy described in Section 3.2. The parameter configurations of all algorithms are summarized in Table 4.

Table 4. Parameter settings for all algorithms.

Algorithm	Key Parameters
A-ALNS [11]	Max iterations 5000, annealing coeff. 0.9975, removal rate 10%, score increments (30,20,10,1), $\phi = 50$, $\rho = 5$, $M = 100$ (Chinese area).
GRILS [20]	$StartGreed = 1$, $GreedyRange = 0.2$, $GreedDecrease = 0.02$, max non-improving iterations 300.
GA	Population size $N_p = 10$, crossover prob. 0.9, mutation prob. 0.1, $MFE = 5000$ (RL-GA without Q-learning).
RL-GA [21]	$T = 1000$, $\alpha = 0.01$, $\gamma = 0.95$, $\epsilon = 0.01$, $L = 2$, $N_p = 10$, $MFE = 5000$.
HCPS	Policy/value net hidden layers 8-512-512, activation Tanh, PPO epochs 10, batch size 256, $\gamma = 0.99$, $\lambda = 0.95$, clip=0.2, KL target 0.01.

Fig. 3 presents the target completion rate (a) and runtime (b, log scale) across all 15 problem scales (100–1500 targets). HCPS achieves 100% completion for instances up to 400 targets. For scales 500 to 1500, HCPS consistently yields the highest completion rates.

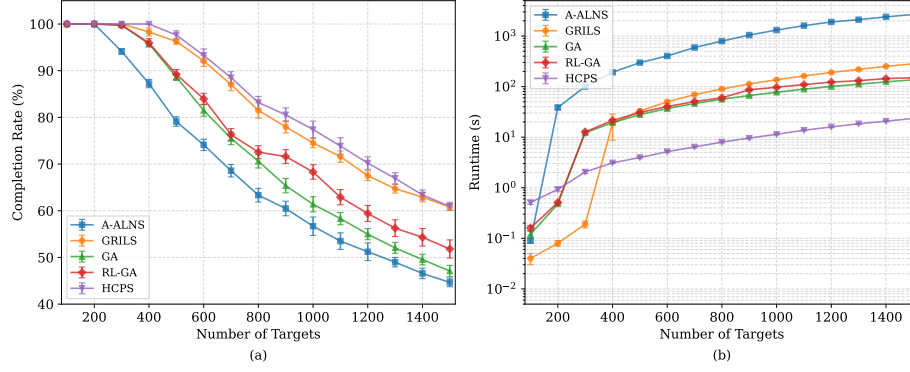


Fig. 3. Comparison of (a) target completion rate and (b) computational time. Error bars indicate standard deviation over 20 test instances per scale.

In terms of computational efficiency, HCPS substantially outperforms all baseline algorithms. For scales exceeding 300, HCPS reduces runtime by an average of 84.8% compared to the second-fastest algorithm (GA) and by 91.0% compared to the algorithm with the second-best completion rate (GRILS). For smaller scales (100–300), where resources are abundant relative to the number of tasks, early termination of heuristic iterations occurs, leading to faster baseline runtimes. Consequently, HCPS exhibits slightly slower solving speeds in these small-scale cases, though the absolute runtime differences remain negligible.

Statistical significance is verified using the Wilcoxon signed-rank test ($p < 0.05$), confirming that HCPS significantly outperforms all baselines in both quality and efficiency for scales ≥ 500 .

Table 5 presents a condensed comparison on four scales (300, 700, 1000, 1500). The complete numerical results (mean $\pm$ standard deviation over 20 test instances per scale) are provided in the supplementary material (Table S2).

Table 5. Selected performance comparison (mean $\pm$ std over 20 test instances per scale).

Scale	Metric	A-ALNS	GRILS	GA	RL-GA	HCPS
300	F_s (%)	94.13 $\pm$ 0.56	100.0 $\pm$ 0.0	99.78 $\pm$ 0.34	99.75 $\pm$ 0.42	100.0 $\pm$ 0.0
	t (s)	100.17 $\pm$ 5.23	0.19 $\pm$ 0.03	12.20 $\pm$ 0.78	12.72 $\pm$ 0.85	2.05 $\pm$ 0.14
700	F_s (%)	68.58 $\pm$ 1.34	87.04 $\pm$ 1.33	75.50 $\pm$ 1.34	76.31 $\pm$ 1.28	88.52 $\pm$ 1.25
	t (s)	593.19 $\pm$ 25.34	69.49 $\pm$ 2.91	46.30 $\pm$ 1.89	50.62 $\pm$ 2.01	6.81 $\pm$ 0.27
1000	F_s (%)	56.69 $\pm$ 1.97	74.51 $\pm$ 1.01	61.38 $\pm$ 1.62	68.31 $\pm$ 1.56	77.39 $\pm$ 1.82
	t (s)	1312.90 $\pm$ 42.23	136.72 $\pm$ 4.23	77.32 $\pm$ 2.45	97.04 $\pm$ 2.89	11.23 $\pm$ 0.42
1500	F_s (%)	44.65 $\pm$ 0.93	60.80 $\pm$ 0.69	47.08 $\pm$ 1.21	51.81 $\pm$ 1.94	60.95 $\pm$ 0.85
	t (s)	2668.85 $\pm$ 80.78	282.62 $\pm$ 6.01	136.44 $\pm$ 3.56	149.70 $\pm$ 4.45	23.28 $\pm$ 0.76

4.4 Ablation Studies

We conduct ablation experiments to quantify the contribution of each component in HCPS, including all engineered features (from CAP) and the hierarchical reward components. The full model (HCPS-Full) includes all features and the complete reward. We systematically remove each component and evaluate the impact on solution quality across three problem scales (700, 1000, 1500 targets). Table 6 reports the relative completion rate (F_s) normalized to the full model (set to 100%) when individual components are omitted.

The results indicate that the primary objective *target completion* has the largest impact, with its removal reducing performance by over 13%. Among the reward components, *kill penalty* also plays a critical role, causing a 6–8% drop when omitted. Among the features, *Target kill count* has the strongest influence. The relative importance of these components remains broadly consistent across problem scales, though the impact of removing components tends to be slightly smaller on larger scales.

Table 6. Ablation results: relative completion rate (F_s) when removing key components (normalized to full model = 100%).

Removed Component	700 targets	1000 targets	1500 targets
None (Full model)	100.00	100.00	100.00
Target completion (reward)	85.47	86.22	86.47
Kill penalty (reward)	92.15	93.44	93.72
Target kill count (feature)	94.60	94.24	95.44
Orbit resource pressure (feature)	97.07	96.85	97.40
Remaining opportunities (feature)	98.26	98.30	98.82
Urgency (reward)	98.46	98.71	99.15
Transition time overhead (feature)	98.65	98.83	99.61
Conflict count (feature)	99.07	99.85	99.79

5 Conclusion and Future Work

This paper proposed HCPS, a hybrid framework that treats domain heuristics as structural shapers of the search space for combinatorial optimization with expensive per-step evaluation. By combining adaptive Top- K pruning and deep feature extraction with PPO policy learning, HCPS reduces per-step complexity while preserving solution quality. Experiments on agile Earth observation satellite scheduling demonstrate that HCPS outperforms state-of-the-art baselines in both quality and efficiency, and ablation studies confirm the contribution of each component.

Future work includes incorporating uncertainty-aware features, extending temporal causality to cross-interval rescheduling, and transferring the HCPS paradigm to other large-scale engineering scheduling problems.

Acknowledgements This work was supported by the National Natural Science Foundation of China (NSFC No.62076031 and No.62076036).

References

1. Chen, J., Chen, M., Wen, J., He, L., Liu, X.: A heuristic construction neural network method for the time-dependent agile earth observation satellite scheduling problem. *Mathematics* **10**(19) (2022). <https://doi.org/10.3390/math10193498>, <https://www.mdpi.com/2227-7390/10/19/3498>
2. Chen, M., Chen, Y., Chen, Y., Qi, W.: Deep reinforcement learning for agile satellite scheduling problem. In: 2019 IEEE Symposium Series on Computational Intelligence (SSCI). pp. 126–132 (2019). <https://doi.org/10.1109/SSCI44817.2019.9002957>
3. Chen, M., Du, Y., Tang, K., Xing, L., Chen, Y., Chen, Y.: Learning to construct a solution for the agile satellite scheduling problem with time-dependent transition times. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* **54**(10), 5949–5963 (2024). <https://doi.org/10.1109/TSMC.2024.3411640>
4. Cui, K., Xiang, J., Zhang, Y.: Mission planning optimization of video satellite for ground multi-object staring imaging. *Advances in Space Research* **61**(6), 1476–1489 (2018). <https://doi.org/https://doi.org/10.1016/j.asr.2017.10.056>, <https://www.sciencedirect.com/science/article/pii/S0273117717307950>
5. Du, B., Li, S., She, Y., Li, W., Liao, H., fei Wang, H.: Area targets observation mission planning of agile satellite considering the drift angle constraint. *Journal of Astronomical Telescopes, Instruments, and Systems* **4**, 047002 – 047002 (2018), <https://api.semanticscholar.org/CorpusID:173179998>
6. Du, J., Jiang, C., Guo, Q., Guizani, M., Ren, Y.: Cooperative earth observation through complex space information networks. *IEEE Wireless Communications* **23**(2), 136–144 (2016). <https://doi.org/10.1109/MWC.2016.7462495>
7. Du, Y., Wang, T., Xin, B., Wang, L., Chen, Y., Xing, L.: A data-driven parallel scheduling approach for multiple agile earth observation satellites. *IEEE Transactions on Evolutionary Computation* **24**(4), 679–693 (2020). <https://doi.org/10.1109/TEVC.2019.2934148>
8. Fransen, R., Bowden, R., Hadfield, S.: Addressing dimensional scaling in reinforcement learning for symbolic locomotion policies through leveraging inductive priors. 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) pp. 5221–5227 (2025), <https://api.semanticscholar.org/CorpusID:283358088>
9. Globus, A., Crawford, J., Lohn, J., Pryor, A.: A comparison of techniques for scheduling earth observing satellites. pp. 836–843 (01 2004)
10. Graepel, T.: Alphago - mastering the game of go with deep neural networks and tree search. In: Machine learning and knowledge discovery in databases (2016)
11. He, L., Liu, X., Laporte, G., Chen, Y., Chen, Y.: An improved adaptive large neighborhood search algorithm for multiple agile satellites scheduling. *Computers & Operations Research* **100**, 12–25 (2018). <https://doi.org/https://doi.org/10.1016/j.cor.2018.06.020>, <https://www.sciencedirect.com/science/article/pii/S0305054818301722>
12. He, Y., Xing, L., Chen, Y., Pedrycz, W., Wang, L., Wu, G.: A generic markov decision process model and reinforcement learning method for scheduling agile earth observation satellites. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* **52**(3), 1463–1474 (2022). <https://doi.org/10.1109/TSMC.2020.3020732>

13. Lemaître, M., Verfaillie, G., Fargier, H., Lang, J., Bataille, N., Lachiver, J.M.: Sharing the use of earth observation satellites. In: 3rd Workshop on planning and scheduling for space (2002) (2002)
14. Lemaître, M., Verfaillie, G., Jouhaud, F., Lachiver, J., Bataille, N.: How to manage the new generation of agile earth observation satellites. In: Proceedings of the International Symposium on Artificial Intelligence, Robotics and Automation in Space. pp. 1–10. Citeseer (2000)
15. Lemaître, M., Verfaillie, G., Jouhaud, F., Lachiver, J.M., Bataille, N.: Selecting and scheduling observations of agile satellites. *Aerospace Science and Technology* **6**(5), 367–381 (2002). [https://doi.org/https://doi.org/10.1016/S1270-9638\(02\)01173-2](https://doi.org/https://doi.org/10.1016/S1270-9638(02)01173-2), <https://www.sciencedirect.com/science/article/pii/S1270963802011732>
16. Li, Y., Xu, M., Wang, R.: Scheduling observations of agile satellites with combined genetic algorithm. In: Third International Conference on Natural Computation (ICNC 2007). vol. 3, pp. 29–33 (2007). <https://doi.org/10.1109/ICNC.2007.652>
17. Liu, C.L., Layland, J.W.: Scheduling algorithms for multiprogramming in a hard-real-time environment. *J. ACM* **20**(1), 46–61 (Jan 1973). <https://doi.org/10.1145/321738.321743>, <https://doi.org/10.1145/321738.321743>
18. Liu, X., Laporte, G., Chen, Y., He, R.: An adaptive large neighborhood search metaheuristic for agile satellite scheduling with time-dependent transition time. *Computers & Operations Research* **86**, 41–53 (2017). <https://doi.org/https://doi.org/10.1016/j.cor.2017.04.006>, <https://www.sciencedirect.com/science/article/pii/S0305054817300977>
19. Peng, G., Dewil, R., Verbeeck, C., Gunawan, A., Xing, L., Vansteenwegen, P.: Agile earth observation satellite scheduling: An orienteering problem with time-dependent profits and travel times. *Comput. Oper. Res.* **111**, 84–98 (2019), <https://api.semanticscholar.org/CorpusID:197507578>
20. Peng, G., Song, G., He, Y., Yu, J., Xiang, S., Xing, L., Vansteenwegen, P.: Solving the agile earth observation satellite scheduling problem with time-dependent transition times. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* **52**(3), 1614–1625 (2022). <https://doi.org/10.1109/TSMC.2020.3031738>
21. Song, Y., Wei, L., Yang, Q., Wu, J., Xing, L., Chen, Y.: RL-ga: A reinforcement learning-based genetic algorithm for electromagnetic detection satellite scheduling problem. *Swarm and Evolutionary Computation* **77**, 101236 (2023). <https://doi.org/https://doi.org/10.1016/j.swevo.2023.101236>, <https://www.sciencedirect.com/science/article/pii/S221065022300010X>
22. Verfaillie, G., Lemaître, M.: Selecting and scheduling observations for agile satellites: Some lessons from the constraint reasoning community point of view. In: Walsh, T. (ed.) *Principles and Practice of Constraint Programming — CP 2001*. pp. 670–684. Springer Berlin Heidelberg, Berlin, Heidelberg (2001)
23. WANG, H., YANG, Z., ZHOU, W., LI, D.: Online scheduling of image satellites based on neural networks and deep reinforcement learning. *Chinese Journal of Aeronautics* **32**(4), 1011–1019 (2019). <https://doi.org/https://doi.org/10.1016/j.cja.2018.12.018>, <https://www.sciencedirect.com/science/article/pii/S1000936119300305>
24. Wang, P., Reinelt, G., Gao, P., Tan, Y.: A model, a heuristic and a decision support system to solve the scheduling problem of an earth observing satellite constellation. *Computers Industrial Engineering* **61**(2), 322–335 (2011). <https://doi.org/https://doi.org/10.1016/j.cie.2011.02.015>, <https://www.sciencedirect.com/science/article/pii/S0360835211000702>, combinatorial Optimizatiion in Industrial Engineering

25. Wang, X., Han, C., Zhang, R., Gu, Y.: Scheduling multiple agile earth observation satellites for oversubscribed targets using complex networks theory. *IEEE Access* **7**, 110605–110615 (2019). <https://doi.org/10.1109/ACCESS.2019.2925704>
26. Wang, X., Wu, G., Xing, L., Pedrycz, W.: Agile earth observation satellite scheduling over 20 years: Formulations, methods, and future directions. *IEEE Systems Journal* **15**(3), 3881–3892 (2021). <https://doi.org/10.1109/JSYST.2020.2997050>
27. Xu, R., Chen, H., Liang, X., Wang, H.: Priority-based constructive algorithms for scheduling agile earth observation satellites with total priority maximization. *Expert Systems with Applications* **51**, 195–206 (2016). <https://doi.org/https://doi.org/10.1016/j.eswa.2015.12.039>, <https://www.sciencedirect.com/science/article/pii/S0957417415008477>